

Funciones en Python

Código reutilizable, entradas y salidas

Clase de Programación

Profesor: Mateo Taito Stambuk

Email: mateo.taitos@edu.uai.cl

17 de Julio, 2026

Objetivos de la Clase

- **Comprender** qué es una función y por qué se usan
- **Definir** funciones sin entradas ni salidas
- **Llamar** funciones desde el programa principal
- **Trabajar** con parámetros (entradas) y `return` (salidas)
- **Reconocer** el scope de variables y la mutabilidad de listas

¿Qué son las Funciones?

Piezas de código reutilizables

Una función es un bloque de código con **nombre propio** que:

- Realiza una **tarea específica**
- Puede ser **llamado** varias veces desde distintos lugares
- Tiene su propio **ámbito** (las variables internas no se mezclan con el resto)

Analogía: Una función es como una **receta de cocina**.

- Ingredientes → entradas (parámetros)
- Pasos → el cuerpo de la función
- Plato terminado → salida (`return`)

¿Por qué usar Funciones?

Sin funciones

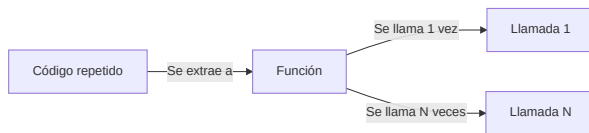
```
print("Hola, Ana")  
print("Hola, Juan")  
print("Hola, Pedro")
```

- Código repetido
- Si cambia el saludo, hay que editar varias líneas
- Difícil de mantener

Con funciones

```
def saludar():  
    print("Hola, bienvenido")  
  
saludar()  
saludar()  
saludar()
```

- Código una sola vez
- Si cambia el saludo, editas una línea
- Fácil de mantener



Estructura de una Función

Anatomía de `def`

```
def saludar():  
    print("Hola, bienvenido a la clase")  
    print("Esta es una función sin entradas ni salidas")
```

`def`

Palabra clave que **define** una función

`saludar`

Nombre de la función (verbos en infinitivo)

`()`

Paréntesis **obligatorios** (entradas)

`:`

Dos puntos + bloque **indentado**

Reglas para Nombrar Funciones

- Usar verbos en infinitivo que describan la acción:
 - `saludar()`, `mostrar_menu()`, `calcular_promedio()`, `imprimir_tabla()`
- Formato snake_case (palabras separadas con guión bajo):
 -  `calcular_promedio`
 -  `CalcularPromedio`, `calcularpromedio`
- Deben empezar con minúscula
- Evitar nombres genéricos como `funcion1`, `proceso`, `hacer_algo`

Convención: Si la función muestra algo → `mostrar_X`. Si calcula algo → `calcular_X`. Si busca algo → `buscar_X` o `encontrar_X`.

Estructura General del Programa

import → funciones → resto del programa

Un archivo `.py` se organiza en tres zonas claras:

```
# ZONA 1: imports (librerías externas)
import random

# ZONA 2: definición de funciones
def saludar():
    print("Hola!")

def mostrar_menu():
    print("1. Jugar")
    print("2. Salir")

# ZONA 3: programa principal
mostrar_menu()
saludar()
```

- Zona 1 (imports): arriba del todo, antes de cualquier código

Diagrama: Estructura del Programa

[Ver diagrama de estructura](#)

Diagrama: Definición de Función

[Ver diagrama de definición](#)

Llamar a una Función

Escribir el nombre con paréntesis

```
def saludar():  
    print("Hola, bienvenido")
```

```
# Llamadas a la función  
saludar()  
saludar()  
saludar()
```

- Línea 1-2: Definimos la función (no hace nada por sí sola)
- Líneas 4, 5, 6: Cada vez que escribimos `saludar()` con paréntesis, ejecutamos el bloque
- La misma función puede llamarse cuantas veces queramos
- Llamar a la función es ejecutar, definir es solo preparar

Trazado de la Ejecución

¿Qué pasa cuando llamamos a la función?

Código:

```
def saludar():  
    print("Hola!")  
    print("Chao!")  
  
print("Inicio")  
saludar()  
print("Fin")
```

Salida en pantalla:

```
Inicio  
Hola!  
Chao!  
Fin
```

El flujo "entra" a la función, ejecuta su bloque, y vuelve al mismo lugar donde se llamó.

Funciones que Llaman a Otras Funciones

```
def saludar():  
    print("Hola!")  
  
def despedir():  
    print("Chao!")  
  
def sesion():  
    saludar()  
    print("Bienvenido a la clase")  
    despedir()  
  
sesion()
```

- `saludar()` y `despedir()` : funciones simples sin entradas ni salidas
- `sesion()` : función que llama a otras funciones
- Cuando `sesion()` se llama, ejecuta `saludar()` , luego `despedir()`
- Una función puede llamar a tantas otras como necesite

Diagrama: Llamada de Función

[Ver diagrama de llamada](#)

Ejemplo 1: Función `saludar()`

Enunciado

Crear un programa que muestre un saludo en pantalla usando una función sin entradas ni salidas.

- La función debe imprimir un mensaje de bienvenida
- La función debe llamarse al menos 3 veces desde el programa principal
- El código debe seguir la estructura: imports → funciones → main

Ejemplo 1: Código Completo

```
# 1. Imports
# (ninguno en este ejemplo)

# 2. Definicion de funciones
def saludar():
    print("Hola, bienvenido a la clase de programacion")
    print("Espero que aprendamos mucho hoy")

# 3. Programa principal
saludar()
saludar()
saludar()
```

Ejemplo 1: Trazado Paso a Paso

Paso	Acción	¿Qué pasa?
1	Python lee el archivo de arriba hacia abajo	Encuentra <code>def saludar()</code> → la define
2	Línea 7: <code>saludar()</code>	Entra a la función
3	Línea 4: <code>print("Hola, ...")</code>	Imprime el primer mensaje
4	Línea 5: <code>print("Espero ...")</code>	Imprime el segundo mensaje
5	Fin de la función	Vuelve a la línea 7
6	Línea 8: <code>saludar()</code>	Repite todo el proceso
7	Línea 9: <code>saludar()</code>	Repite una tercera vez

Entradas y Salidas: Concepto

¿Cómo una función recibe datos y devuelve resultados?

Entradas (parámetros)

Información que la función recibe para trabajar.

```
def saludar(nombre):  
    print(f"Hola, {nombre}")
```

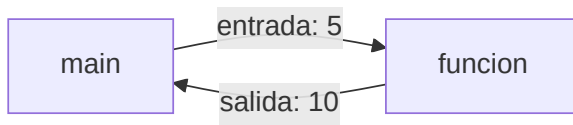
`nombre` es el parámetro: el dato que llega desde afuera.

Salidas (`return`)

Resultado que la función devuelve al llamador.

```
def doble(numero):  
    return numero * 2
```

`return` devuelve un valor para ser usado fuera.



print vs return

¡No son lo mismo!

print

Muestra un mensaje en pantalla.

```
def saludar(nombre):  
    print(f"Hola, {nombre}")
```

- La función no devuelve nada
- Solo imprime
- No se puede guardar el resultado

return

Devuelve un valor al llamador.

```
def saludar(nombre):  
    return f"Hola, {nombre}"
```

- La función devuelve un string
- Se puede guardar: `msg = saludar("Ana")`
- Se puede usar en una expresión

Regla práctica: Si la función **calcula** algo y otro código necesita ese resultado → usa `return`. Si solo quieres **mostrar** un mensaje → usa `print`.

Parámetros: La Entrada de la Función

Sintaxis

```
def saludar(nombre):  
    print(f"Hola, {nombre}")
```

- `nombre` es el parámetro de la función
- Es una variable local que existe solo dentro de la función
- Se inicializa en el momento de la llamada
- Al terminar la función, desaparece

Pasar Argumentos en la Llamada

```
def saludar(nombre, edad):  
    print(f"Hola, {nombre}, tienes {edad} años")  
  
# Llamadas con argumentos posicionales  
saludar("Ana", 20)  
saludar("Juan", 25)  
saludar("Sofia", 22)
```

- `nombre` y `edad` son los parámetros (en la definición)
- `"Ana"` y `20` son los argumentos (en la llamada)
- Los argumentos se asignan en orden a los parámetros
- Se pueden tener tantos parámetros como la función necesite

Múltiples Parámetros

Ejemplo con dos y tres entradas

Dos parámetros:

```
def calcular_area(base, altura):  
    area = base * altura  
    print(f"Area: {area}")  
  
calcular_area(5, 3)  
# Imprime: Area: 15
```

Tres parámetros:

```
def crear_mensaje(nombre, accion, objeto):  
    print(f"{nombre} {accion} {objeto}")  
  
crear_mensaje("Ana", "compra", "pan")  
# Imprime: Ana compra pan
```

Or importante: El orden de los argumentos debe coincidir con el orden de los parámetros.

```
crear_mensaje("compra", "Ana", "pan")  
# Imprime: compra Ana pan (¡incorrecto!)
```

Variables Escalares: Paso por Valor

Las variables escalares (números, strings) no se modifican fuera

```
def intentar_cambiar(x):  
    x = 100  
    print(f"Dentro: {x}")  
  
numero = 5  
intentar_cambiar(numero)  
print(f"Fuera: {numero}")
```

Salida:

```
Dentro: 100  
Fuera: 5
```

- Dentro de la función, `x` vale `100`
- Fuera, `numero` sigue valiendo `5`
- La función recibe una copia, no el original
- Modificar la copia no afecta al original

Diagrama: Flujo de Entradas y Salidas

Ver diagrama de entradas y salidas

La Sentencia `return`

Devolver un valor al llamador

```
def doble(numero):  
    return numero * 2  
  
resultado = doble(7)  
print(resultado)  
print(doble(7) + doble(3))
```

- Línea 1-2: La función devuelve el doble
- Línea 4: `return numero * 2` devuelve `14`
- Línea 4: `resultado` guarda ese `14`
- Línea 6: Podemos usar el valor de `return` directamente en expresiones

Usar el Valor Devuelto

Tres formas equivalentes

1. Guardar

```
resultado = doble(5)
print(resultado)
```

2. Usar directo

```
print(doble(5))
```

3. En expresión

```
total = doble(5) + doble(3)
```

`return` también termina la función: cualquier código después de `return` no se ejecuta.

```
def ejemplo():
    return 10
    print("Esto NUNCA se imprime")
```

Múltiples return

Retornos condicionales

```
def clasificar_edad(edad):  
    if edad < 0:  
        return "Edad invalida"  
    if edad < 18:  
        return "Menor de edad"  
    if edad < 65:  
        return "Adulto"  
    return "Adulto mayor"  
  
print(clasificar_edad(15))  
print(clasificar_edad(30))  
print(clasificar_edad(70))
```

- Cada `return` devuelve un valor distinto
- Solo se ejecuta uno (el primero que coincida con la condición)
- Los demás `return` no se evalúan
- Sirve para manejar casos diferentes

¿Qué pasa si no hay `return`?

La función devuelve `None`

```
def saludar(nombre):  
    print(f"Hola, {nombre}")  
  
resultado = saludar("Ana")  
print(resultado)
```

Salida:

```
Hola, Ana  
None
```

- La función no tiene `return` explícito
- Python devuelve `None` automáticamente
- `None` significa "nada" o "ausencia de valor"

Ejemplo 2: `calcular_promedio()`

Enunciado

Crear una función que reciba una lista de notas y devuelva el promedio.

- La función recibe una lista como entrada
- Calcula el promedio (suma / cantidad)
- Devuelve el resultado con `return`
- Se debe llamar desde el programa principal y mostrar el resultado

Ejemplo 2: Código Completo

```
# 1. Definicion de funciones
def calcular_promedio(notas):
    suma = 0
    for nota in notas:
        suma = suma + nota
    promedio = suma / len(notas)
    return promedio

# 2. Programa principal
notas_alumno = [5.5, 6.0, 4.8, 7.0, 5.2]
promedio_final = calcular_promedio(notas_alumno)
print(f"El promedio es: {promedio_final:.2f}")
```

Ejemplo 2: Trazado Paso a Paso

Datos: `notas = [5.5, 6.0, 4.8, 7.0, 5.2]`

Cálculo:

Paso	Operación	Valor
1	<code>suma = 0</code>	<code>0</code>
2	<code>suma + 5.5</code>	<code>5.5</code>
3	<code>suma + 6.0</code>	<code>11.5</code>
4	<code>suma + 4.8</code>	<code>16.3</code>
5	<code>suma + 7.0</code>	<code>23.3</code>
6	<code>suma + 5.2</code>	<code>28.5</code>

Ejemplo 2: Usos del Valor Devuelto

```
notas_curso = [5.5, 6.0, 4.8, 7.0, 5.2]

# Uso 1: guardar en variable
prom = calcular_promedio(notas_curso)
print(f"Promedio: {prom:.2f}")

# Uso 2: usar en una condicion
if calcular_promedio(notas_curso) >= 4.0:
    print("Aprueba")

# Uso 3: pasar a otra funcion
print(f"Max: {max(notas_curso)}")
print(f"Prom: {calcular_promedio(notas_curso):.2f}")
```

Gracias al `return`, el valor de la función se comporta como cualquier otro valor: se guarda, se compara, se mezcla con otras operaciones.

Característica 1: Scope de Variables

Las variables creadas dentro de la función solo existen allí

```
def mi_funcion():  
    mensaje = "Hola desde adentro"  
    print(mensaje)  
  
mi_funcion()  
print(mensaje) # ERROR
```

- Línea 1-3: `mensaje` se crea dentro de la función
- Línea 5: Al llamar a la función, `mensaje` existe
- Línea 8: Fuera de la función, `mensaje` no existe → `NameError`

Esto se llama "scope local": las variables definidas en una función están aisladas del resto del programa.

Scope Local en Acción

```
def funcion_a():  
    x = 10  
    print(f"En A: {x}")  
  
def funcion_b():  
    x = 20  
    print(f"En B: {x}")  
  
funcion_a()  
funcion_b()  
print(x) # ERROR: x no existe fuera
```

Salida:

```
En A: 10  
En B: 20  
NameError: name 'x' is not defined
```

- Cada función tiene su propia variable `x`
- Son independientes: la `x` de A no afecta a la `x` de B
- Fuera de las funciones, ninguna `x` existe

Diagrama: Scope de Variables

[Ver diagrama de scope](#)

Característica 2: Mutabilidad de Listas

Las listas externas sí se pueden modificar dentro de la función

```
def agregar_elemento(lista, elemento):  
    lista.append(elemento)  
    print(f"Dentro: {lista}")  
  
mis_notas = [5.5, 6.0]  
agregar_elemento(mis_notas, 7.0)  
print(f"Fuera: {mis_notas}")
```

Salida:

```
Dentro: [5.5, 6.0, 7.0]  
Fuera: [5.5, 6.0, 7.0]
```

- La lista se modifica dentro de la función
- Y el cambio persiste afuera
- Esto es distinto a las variables escalares (que no se modifican)

¿Por qué pasa esto?

Listas vs Escalares: la diferencia clave

Escalares (números, strings)

```
def cambiar(x):  
    x = 100  
  
n = 5  
cambiar(n)  
print(n) # 5
```

Se pasa una copia.

Listas

```
def agregar(lista):  
    lista.append(99)  
  
nums = [1, 2]  
agregar(nums)  
print(nums) # [1, 2, 99]
```

Se pasa una referencia.

Analogía: Un escalar es como una **copia de un papel**: lo que escribas en la copia no afecta al original. Una lista es como una **dirección de una casa**: si modificas la casa, el cambio es visible para todos los que tengan la dirección.

Diagrama: Mutabilidad de Listas

[Ver diagrama de mutabilidad](#)

Resumen: Comportamiento por Tipo

Tipo de dato	Se modifica fuera?	Razón
<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>	No	Se pasa una copia
<code>list</code> , <code>dict</code>	Sí	Se pasa una referencia

Consecuencia práctica: Si no quieres que la función modifique tu lista original, pasa una copia:

```
agregar_elemento(mis_notas[:], 7.0) # [:] crea una copia
```

Ejercicio 1: Mostrar Menú

Enunciado del Problema

Un programa de consola debe mostrar un menú con tres opciones cada vez que se necesita.

```
===== MENU PRINCIPAL =====  
1. Jugar  
2. Ver puntaje  
3. Salir  
=====
```

Misión: Crear una función `mostrar_menu()` que imprima el menú, y llamarla al menos dos veces en el programa principal.

Ejercicio 1: Planteamiento

Variables

- No hay variables externas
- La función no recibe parámetros
- La función no devuelve nada

Lógica

- `def mostrar_menu():`
- Imprimir líneas con `print`
- Usar `print` (no `return`, porque solo queremos mostrar)

Salida esperada (al llamar la función 2 veces): el menú se imprime completo, dos veces.

Ejercicio 1: Diagrama de Flujo

Ejercicio 1: Código Completo

```
# 1. Definicion de la funcion
def mostrar_menu():
    print("===== MENU PRINCIPAL =====")
    print("1. Jugar")
    print("2. Ver puntaje")
    print("3. Ver configuracion")
    print("4. Salir")
    print("=====")

# 2. Programa principal
mostrar_menu()
print("Cargando datos del jugador... ")
mostrar_menu()
```

Ejercicio 1: Explicación Detallada

- Líneas 1-9: Definimos `mostrar_menu()` . No tiene parámetros ni `return` . Solo imprime.
- Línea 3: `=====` decorativos para que el menú se vea ordenado.
- Líneas 4-7: Cada `print` muestra una opción numerada.
- Línea 8: Línea decorativa final.
- Línea 11: Primera llamada al menú (aparece arriba de "Cargando datos...").
- Línea 12: Una acción intermedia (no es la función, es código normal).
- Línea 13: Segunda llamada. El menú aparece idéntico porque es la misma función.

Ejercicio 1: Revisar

Caso de prueba: 2 llamadas

Salida esperada:

```
===== MENU PRINCIPAL =====  
1. Jugar  
2. Ver puntaje  
3. Ver configuracion  
4. Salir  
=====  
Cargando datos del jugador...  
===== MENU PRINCIPAL =====  
1. Jugar  
2. Ver puntaje  
3. Ver configuracion  
4. Salir  
=====
```

El menú aparece completo, dos veces, separadas por el mensaje intermedio.

Ejercicio 2: Saludar Persona

Enunciado del Problema

Necesitamos una función que reciba un nombre y una edad, e imprima un saludo personalizado.

```
Hola Ana, tienes 20 años. Bienvenido a la clase.  
Hola Juan, tienes 25 años. Bienvenido a la clase.
```

Misión: Crear `saludar_persona(nombre, edad)` que use f-strings y llamarla con al menos 3 personas distintas.

Ejercicio 2: Planteamiento

Variables (parámetros)

- `nombre` : string con el nombre
- `edad` : número entero

Lógica

- `def saludar_persona(nombre, edad):`
- Construir el mensaje con f-string
- Imprimir el resultado con `print`

Diferencia con Ejercicio 1: Esta función sí recibe entradas (parámetros), pero no devuelve nada (usa `print`).

Ejercicio 2: Diagrama de Flujo

Ejercicio 2: Código Completo

```
# 1. Definicion de la funcion
def saludar_persona(nombre, edad):
    print(f"Hola {nombre}, tienes {edad} años. Bienvenido a la clase.")

# 2. Programa principal
saludar_persona("Ana", 20)
saludar_persona("Juan", 25)
saludar_persona("Sofia", 22)
```

Ejercicio 2: Explicación Detallada

- Línea 1-2: `def saludar_persona(nombre, edad):` define la función con dos parámetros.
- Línea 2: f-string con `{nombre}` y `{edad}` : los placeholders se reemplazan con los valores recibidos.
- Línea 5: `saludar_persona("Ana", 20)` → `"Ana"` se asigna a `nombre` , `20` se asigna a `edad` .
- Líneas 6, 7: Las llamadas con `"Juan", 25` y `"Sofia", 22` repiten el proceso con otros valores.
- Convención: Los parámetros van en la definición (`nombre, edad`). Los argumentos van en la llamada (`"Ana", 20`).

Ejercicio 2: Revisar

Trazado de las llamadas

Datos de prueba:

Llamada	nombre	edad	Salida
<code>saludar_persona("Ana", 20)</code>	<code>"Ana"</code>	<code>20</code>	<code>Hola Ana, tienes 20 años ...</code>
<code>saludar_persona("Juan", 25)</code>	<code>"Juan"</code>	<code>25</code>	<code>Hola Juan, tienes 25 años ...</code>
<code>saludar_persona("Sofia", 22)</code>	<code>"Sofia"</code>	<code>22</code>	<code>Hola Sofia, tienes 22 años ...</code>

La misma función produce mensajes distintos según los argumentos.

Ejercicio 3: Encontrar el Mayor

Enunciado del Problema

Necesitamos una función que reciba una lista de números y devuelva el mayor de todos.

```
numeros = [4, 2, 8, 1, 9, 5, 3]
mayor = encontrar_mayor(numeros)
print(mayor) # 9
```

Misión: Crear `encontrar_mayor(lista)` que use `return` para devolver el valor, y probarla con al menos 3 listas distintas.

Ejercicio 3: Planteamiento

Variables (parámetro)

- `numeros` : lista de enteros o flotantes
- Variable local: `mayor_actual` para 跟踪

Lógica

- Empezar con `mayor = numeros[0]`
- Recorrer la lista con `for`
- Si `numeros[i] > mayor`, actualizar
- `return mayor` al final

Aquí sí usamos `return` : la función calcula un valor y el llamador lo usa.

Ejercicio 3: Diagrama de Flujo

Ejercicio 3: Código Completo

```
# 1. Definicion de la funcion
def encontrar_mayor(numeros):
    mayor = numeros[0]
    for i in range(1, len(numeros)):
        if numeros[i] > mayor:
            mayor = numeros[i]
    return mayor

# 2. Programa principal
lista_a = [4, 2, 8, 1, 9, 5, 3]
lista_b = [10, 5, 7]
lista_c = [-3, -10, -1, -5]

print(f"Mayor de A: {encontrar_mayor(lista_a)}")
print(f"Mayor de B: {encontrar_mayor(lista_b)}")
print(f"Mayor de C: {encontrar_mayor(lista_c)}")
```

Ejercicio 3: Explicación Detallada

- Línea 1-2: Definimos `encontrar_mayor(numeros)` . Recibe una lista.
- Línea 3: Inicializamos `mayor` con el primer elemento (será nuestra referencia inicial).
- Línea 4: `range(1, len(numeros))` : empezamos desde la posición 1, porque la 0 ya la usamos.
- Línea 5-6: Si el elemento actual es mayor que `mayor` , lo actualizamos.
- Línea 8: `return mayor` devuelve el valor encontrado.
- Línea 11-13: Tres listas distintas, todas se procesan con la misma función.
- Línea 15-17: Usamos el valor de retorno directamente dentro de f-strings.

Ejercicio 3: Revisar

Trazado con `lista_a = [4, 2, 8, 1, 9, 5, 3]`

Iteración	i	numeros[i]	mayor antes	mayor después
Inicio	-	-	-	4
1	1	2	4	4
2	2	8	4	8
3	3	1	8	8
4	4	9	8	9
5	5	5	9	9
6	6	3	9	9

Resumen de la Clase

Conceptos fundamentales

Funciones básicas

- Definir con `def nombre():`
- Llamar con `nombre()`
- Bloque indentado
- Tres zonas: `import` → funciones → `main`

Entradas y salidas

- Parámetros: entradas (entre paréntesis)
- `return` : valor devuelto
- Múltiples parámetros permitidos
- `print` muestra, `return` devuelve

Scope y mutabilidad

- Variables locales: solo dentro de la función
- Escalares: no se modifican fuera
- Listas: sí se modifican fuera
- Usar `lista[:]` para evitarlo

Conceptos Reforzados

Lo que practicamos hoy

- Estructura del programa: imports → funciones → main
- Definir y llamar funciones sin entradas ni salidas
- Parámetros y `return` para funciones con entradas y salidas
- Diferencia entre `print` y `return`
- Scope de variables locales y la mutabilidad de listas externas
- Diagramas de flujo para planificar la lógica de cada función

Regla de oro: Una buena función hace **una sola cosa**, tiene un nombre claro (verbo) y, si calcula algo, lo devuelve con `return` .

layout: center class: text-center

¡Gracias!

¿Preguntas?

Funciones en Python